

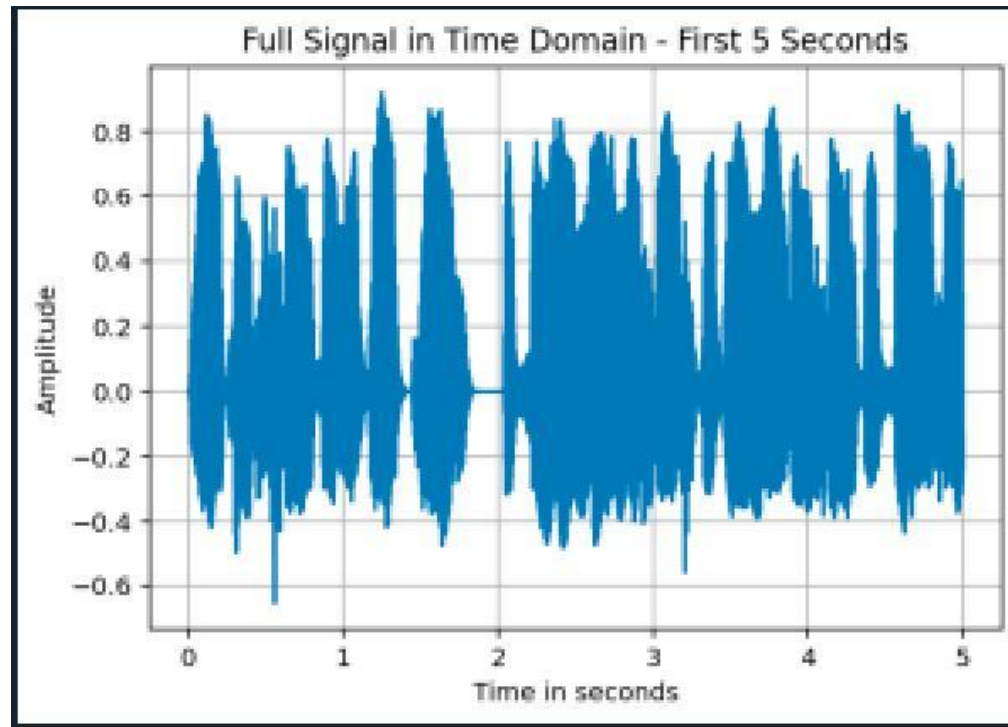
SIGNALS LAB PROJECT

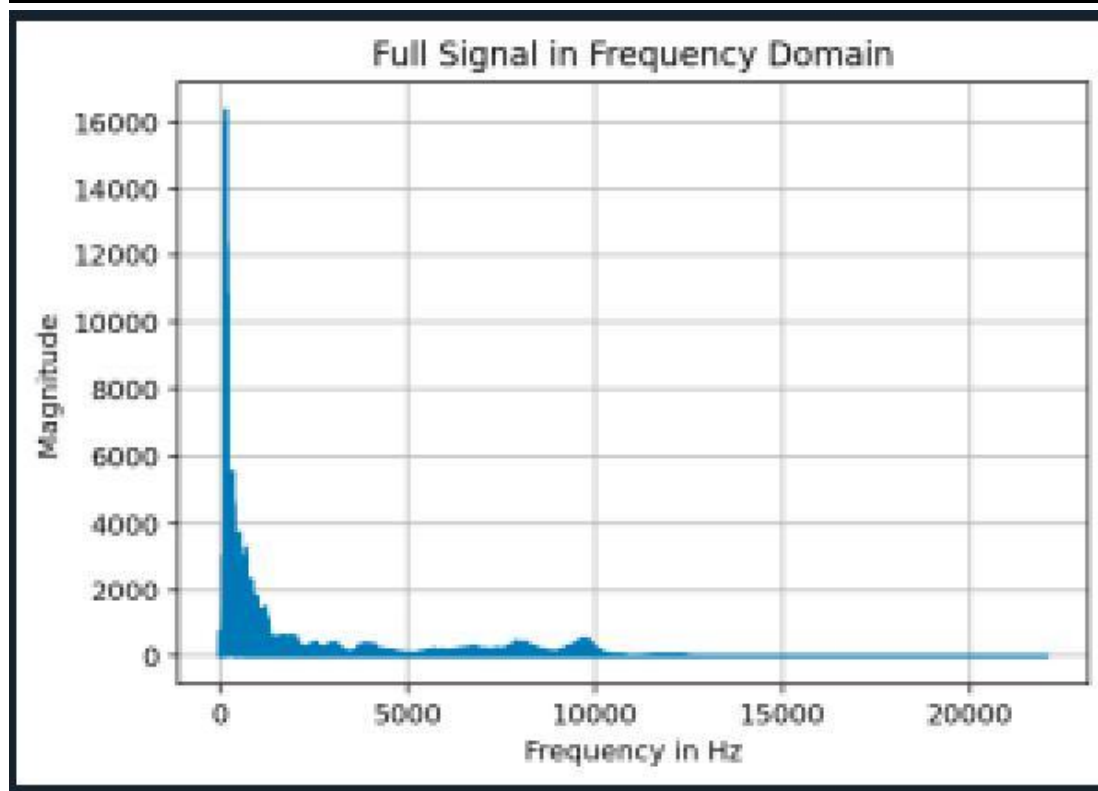
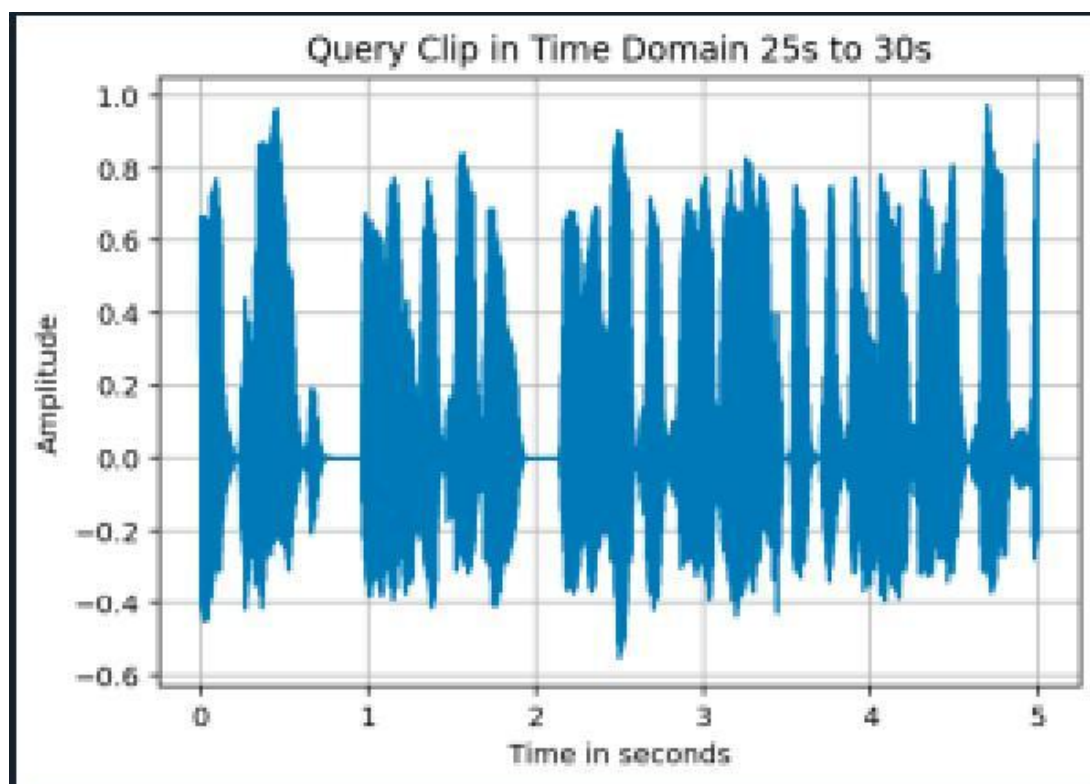
T-24

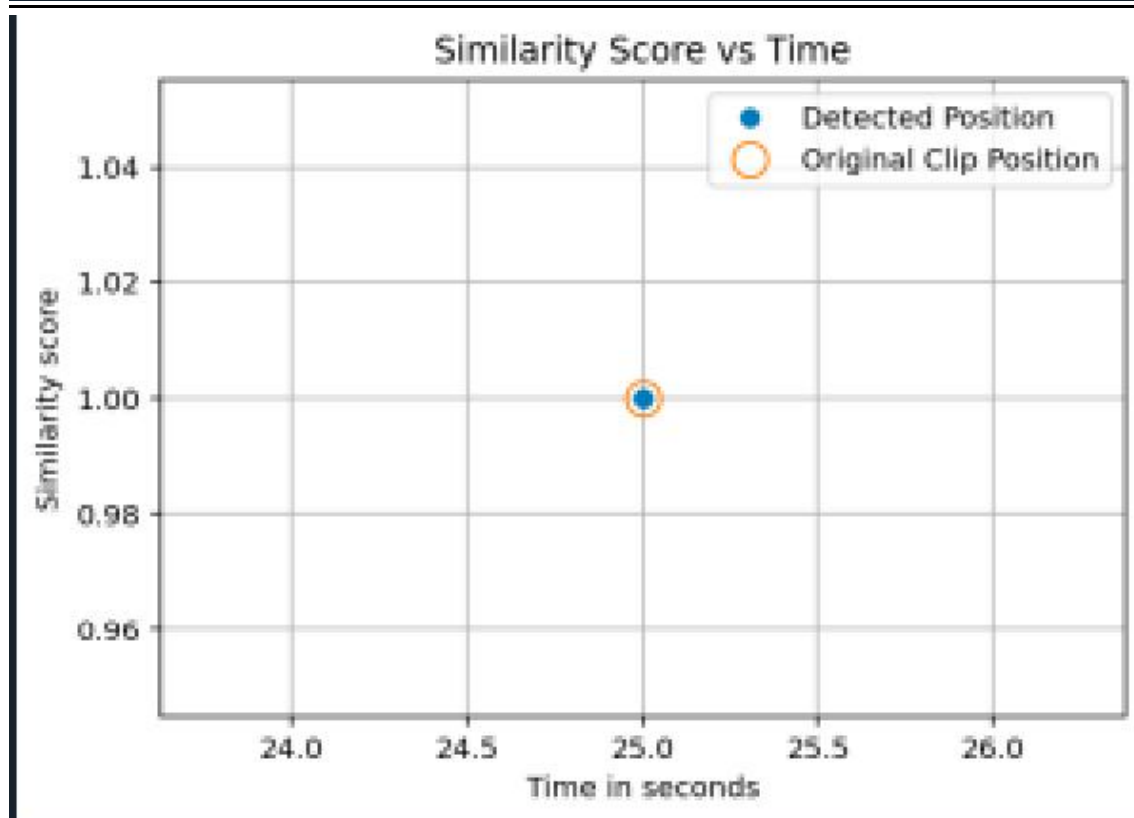
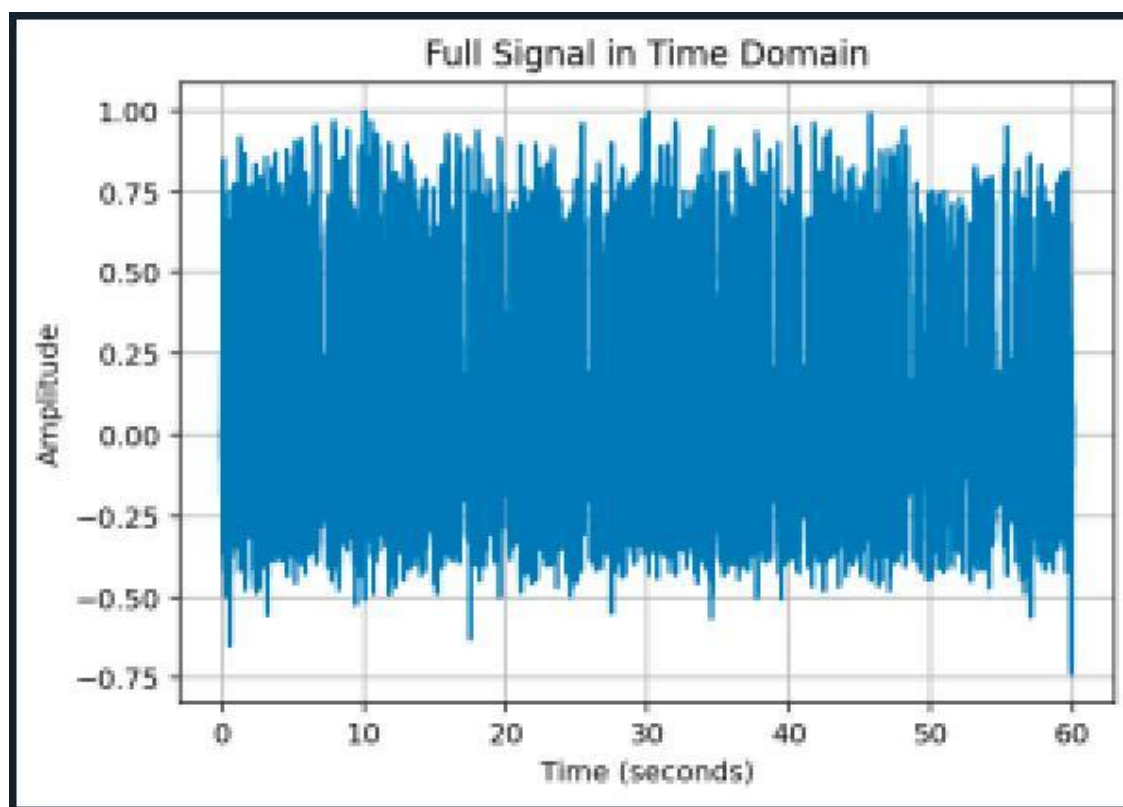
OMAR MOATAZ EL SOBKY 64-3084

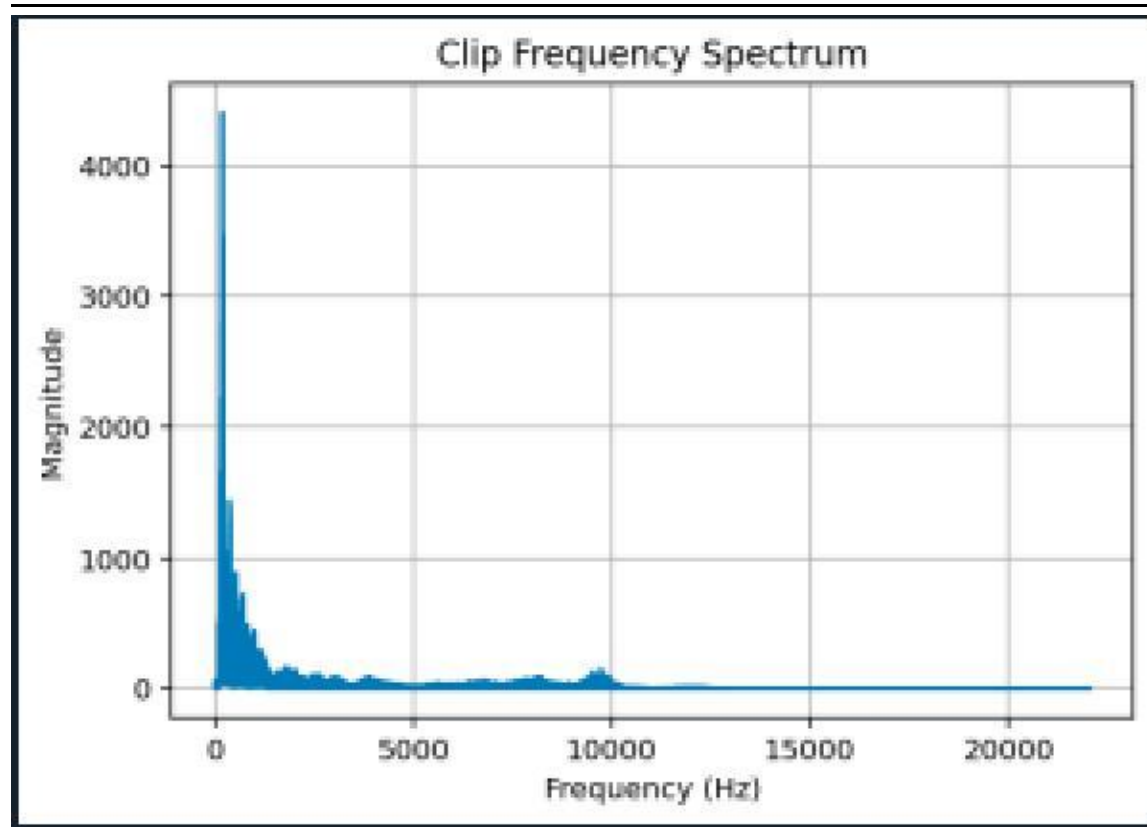
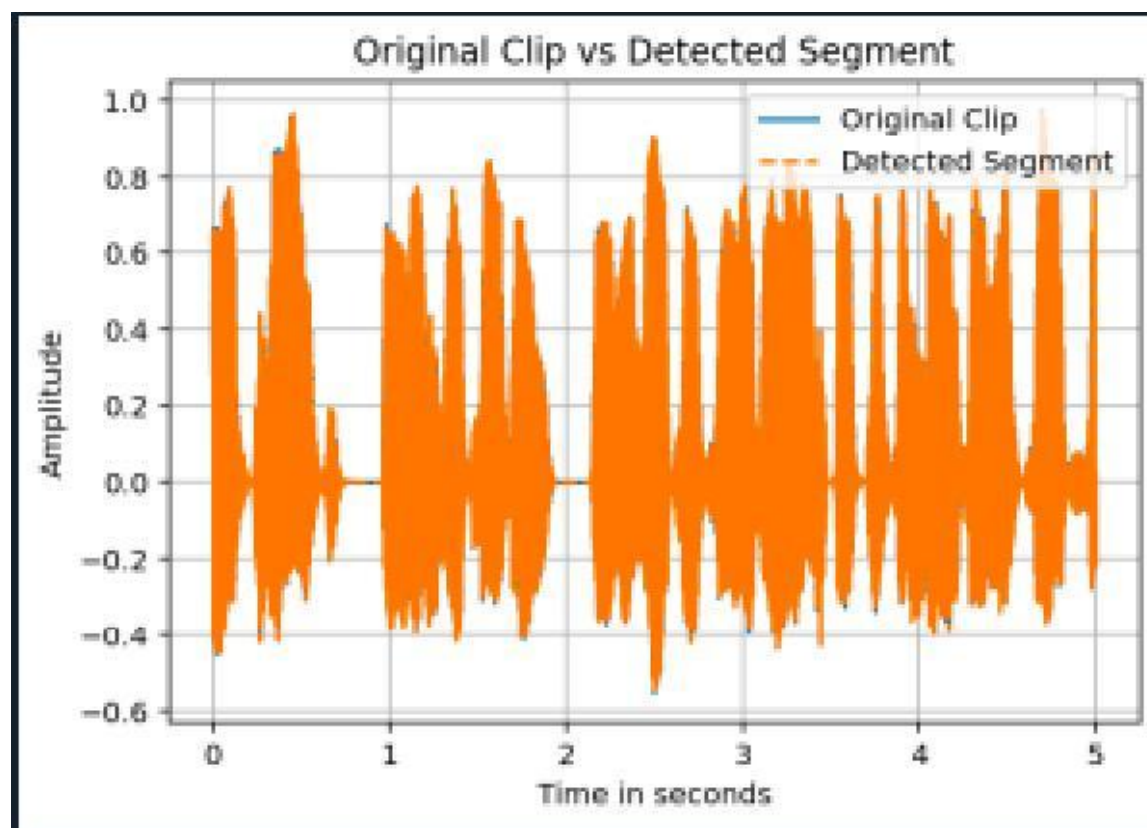
HAMZA WAEEL SALLAM 64-5469

GRAPHS:









CODE:

```
import numpy as np
from scipy.fft import fft
from scipy.io import wavfile
import matplotlib.pyplot as plt

audio_file = "/Users/hamzasallam/audio.wav"
sampling_frequency, full_signal = wavfile.read(audio_file)

print("Sampling frequency:", sampling_frequency, "Hz")
print("Original signal shape:", full_signal.shape)
print("Original signal data type:", full_signal.dtype)

if len(full_signal.shape) == 2:
    full_signal = np.mean(full_signal, axis=1)

full_signal = full_signal.astype(float)
full_signal = full_signal / np.max(np.abs(full_signal))

print("Processed signal shape:", full_signal.shape)
print("Processed signal data type:", full_signal.dtype)
print("Length of full signal in samples:", len(full_signal))
```

```
print("Length of full signal in seconds:", len(full_signal) / sampling_frequency)
```

```
time_axis = np.arange(len(full_signal)) / sampling_frequency
```

```
plt.figure()
```

```
plt.plot(time_axis, full_signal)
```

```
plt.xlabel("Time (seconds)")
```

```
plt.ylabel("Amplitude")
```

```
plt.title("Full Signal in Time Domain")
```

```
plt.grid(True)
```

```
plt.show()
```

```
seconds_to_plot = 5
```

```
samples_to_plot = seconds_to_plot * sampling_frequency
```

```
plt.figure()
```

```
plt.plot(time_axis[:samples_to_plot], full_signal[:samples_to_plot])
```

```
plt.xlabel("Time in seconds")
```

```
plt.ylabel("Amplitude")
```

```
plt.title("Full Signal in Time Domain - First 5 Seconds")
```

```
plt.grid()
```

```
plt.show()
```

```
clip_start_time = 25
clip_duration = 5
clip_start_sample = int(clip_start_time * sampling_frequency)
clip_end_sample = int((clip_start_time + clip_duration) * sampling_frequency)
clip = full_signal[clip_start_sample:clip_end_sample]
```

```
print("Original clip position:", clip_start_time, "seconds")
print("Clip duration:", clip_duration, "seconds")
print("Clip length in samples:", len(clip))
print("Clip starts at sample:", clip_start_sample)
print("Clip ends at sample:", clip_end_sample)
```

```
clip_time_axis = np.arange(len(clip)) / sampling_frequency
```

```
plt.figure()
plt.plot(clip_time_axis, clip)
plt.xlabel("Time in seconds")
plt.ylabel("Amplitude")
plt.title("Query Clip in Time Domain 25s to 30s")
plt.grid()
plt.show()
```

```
def compute_fft_magnitude(signal_segment):
    fft_result = fft(signal_segment)
```

```
half_length = len(fft_result) // 2
one_sided_fft = fft_result[:half_length]
magnitude = np.abs(one_sided_fft)
return magnitude
```

```
full_signal_fft_magnitude = compute_fft_magnitude(full_signal)
full_frequency_axis = np.linspace(0,sampling_frequency /
2,len(full_signal_fft_magnitude))
```

```
plt.figure()
plt.plot(full_frequency_axis, full_signal_fft_magnitude)
plt.xlabel("Frequency in Hz")
plt.ylabel("Magnitude")
plt.title("Full Signal in Frequency Domain")
plt.grid()
plt.show()
```

```
clip_fft_magnitude = compute_fft_magnitude(clip)
```

```
print("Clip FFT length:", len(clip_fft_magnitude))
print("First 10 FFT magnitude values:")
print(clip_fft_magnitude[:10])
```



```
frequency_axis = np.linspace(0, sampling_frequency / 2, len(clip_fft_magnitude))
```

```
plt.figure()
```

```
plt.plot(frequency_axis, clip_fft_magnitude)
```

```
plt.xlabel("Frequency (Hz)")
```

```
plt.ylabel("Magnitude")
```

```
plt.title("Clip Frequency Spectrum")
```

```
plt.grid(True)
```

```
plt.show()
```

```
window_length = len(clip)
```

```
step_size = int(0.1 * sampling_frequency)
```

```
similarity_scores = []
```

```
time_positions = []
```

```
for start_sample in range(0, len(full_signal) - window_length, step_size):
```

```
    window_segment = full_signal[start_sample:start_sample + window_length]
```

```
    window_fft_magnitude = compute_fft_magnitude(window_segment)
```

```
    numerator = np.dot(clip_fft_magnitude, window_fft_magnitude)
```

```
    denominator = np.linalg.norm(clip_fft_magnitude) *  
np.linalg.norm(window_fft_magnitude)
```

```
    similarity = numerator / denominator
```

```
    similarity_scores.append(similarity)
```

```
    time_positions.append(start_sample / sampling_frequency)
```

```
similarity_scores = np.array(similarity_scores)
```

```
time_positions = np.array(time_positions)
print("Audio matching completed.")
print("Number of windows checked:", len(similarity_scores))

best_match_index = np.argmax(similarity_scores)
detected_position = time_positions[best_match_index]
best_similarity_score = similarity_scores[best_match_index]

print("Original clip position:", clip_start_time, "seconds")
print("Detected position:", detected_position, "seconds")
print("Best similarity score:", best_similarity_score)

plt.figure()
plt.plot(detected_position, best_similarity_score, 'o', label="Detected Position")
plt.plot(clip_start_time, best_similarity_score, 'o', markerfacecolor= 'none',
markersize=12,label="Original Clip Position")
plt.xlabel("Time in seconds")
plt.ylabel("Similarity score")
plt.title("Similarity Score vs Time")
plt.legend()
plt.grid()
plt.show()

detected_start_sample = int(detected_position * sampling_frequency)
detected_end_sample = detected_start_sample + len(clip)
```

```
detected_segment = full_signal[detected_start_sample:detected_end_sample]  
clip_time_axis = np.arange(len(clip)) / sampling_frequency
```

```
plt.figure()  
plt.plot(clip_time_axis, clip, label="Original Clip")  
plt.plot(clip_time_axis, detected_segment, linestyle='--', label="Detected  
Segment")  
plt.xlabel("Time in seconds")  
plt.ylabel("Amplitude")  
plt.title("Original Clip vs Detected Segment")  
plt.legend()  
plt.grid()  
plt.show()
```